GRUS

PRIVATE MESSAGING PLATFORM

Complete Software Project Documentation

Private by Design. Connected in Real Time.

Prepared for: Project Owner, Development and Technical Review

Author / Copyright Owner: Harsha Vardhan Goud

Version 1.0 | 05 November 2026

Live frontend: https://grus-429e.onrender.com/
Backend: https://grus-messenger.onrender.com
Health: https://grus-messenger.onrender.com/api/health

Document Control

Field	Details
Project	GRUS – Private Messaging Platform
Document	Software project overview, requirements, architecture, design, deployment, testing and operations
Version / Date	1.0 / 05 November 2026
Owner	Harsha Vardhan Goud
Evidence basis	Development conversation, shared source-tree inventory, build/deployment logs and described behavior

Scope and evidence note

This document consolidates project details shared during development. It is not a complete line-by-line audit of the current repository. Items whose implementation, schema, security properties or test status have not been independently verified are explicitly described as requiring confirmation. Do not treat planned cryptographic features as audited production guarantees.

Contents

- 1. Executive summary and product vision
- 2. Goals, scope and roles
- 3. Functional requirements and features
- 4. Technology stack and architecture
- 5. Frontend design
- 6. Backend design
- 7. Data, API and real-time communication
- 8. Security and cryptographic design
- 9. Deployment and configuration
- 10. Installation and developer workflow
- 11. Testing and QA
- 12. Monitoring and troubleshooting
- 13. Risks and verification items
- 14. Roadmap, glossary and ownership

1. Executive Summary

GRUS is a browser-based private messaging platform intended to support account access, user discovery and connections, one-to-one conversations, real-time message delivery, message history and conversation controls. It uses a React/Vite frontend, Python/Flask backend, MongoDB persistence and Socket.IO real-time communication.

Product vision

Your Conversations. Your Privacy. Your Control.

- Provide a private one-to-one messaging experience.
- Support registration, login and password recovery.
- Allow user discovery and connection requests.
- Deliver messages and updates in real time.
- Retain conversation history and provide message controls.
- Use client-side cryptographic mechanisms where implemented and verified.
- Deploy frontend and backend independently.

Service	Current URL
Frontend	https://grus-429e.onrender.com/
Backend	https://grus-messenger.onrender.com
Health endpoint	https://grus-messenger.onrender.com/api/health
Repository	https://github.com/Aluval/grus-messenger

2. Goals, Scope and User Roles

In-scope capabilities

- Registration and login; password recovery and recovery setup.
- User discovery, connection requests and connection management.
- One-to-one conversations and message history.
- Real-time communication using Socket.IO.
- Frontend cryptographic service, identity fingerprint and identity restore modules present in the project inventory.
- Message selection, bulk deletion, delete-for-me/everyone and clear-conversation work described during development.

Roles

Role	Responsibilities
Guest	View public landing/authentication screens; register or sign in.
Registered user	Manage account, discover users, connect, message and manage conversations.
System operator	Manage deployment, secrets, database access, monitoring and releases.

Not confirmed

Group chat, voice/video calls, push notifications, compliance certifications, penetration-test results and high-availability guarantees are not established by the available evidence.

3. Functional Requirements and Features

ID	Feature	Expected behavior
FR-01	Registration	New user submits details and receives validation/success response.
FR-02	Login	Registered user authenticates and accesses protected areas.
FR-03	Recovery	User initiates and completes configured password recovery.
FR-04	Discovery	Signed-in user finds other users according to discovery rules.
FR-05	Connections	Users manage connection requests and relationships.
FR-06	Private chat	Eligible users open a one-to-one conversation.
FR-07	Send message	Client encrypts content before API submission where encryption flow is active.
FR-08	History	Prior messages load and decrypt when the correct key is available.
FR-09	Realtime	Connected clients receive authorized Socket.IO updates.
FR-10	Status	Delivery/read acknowledgements work where supported.
FR-11	Blocking	Block/unblock is enforced by backend and socket paths.
FR-12	Deletion	Message and conversation deletion obeys selected scope and authorization.
FR-13	Identity	Identity key/fingerprint and restore flows operate as implemented.
FR-14	Attachments	File/image selection, storage and encryption behavior is validated.

Message deletion requirements under development

- Individual message checkboxes.
- Select All, including messages beyond the latest 100.
- Delete for Me and Delete for Everyone.
- Clear Conversation clears only the current user's history when that is the selected behavior.
- Real-time synchronization after bulk deletion or clearing.

Deletion scope must be explicit. Delete-for-me must not remove another participant's copy; delete-for-everyone requires authorization and synchronized propagation.

4. Technology Stack and Architecture

Layer	Technology	Purpose
Frontend	React.js	UI and client-side state.
Build	Vite	Development and production asset bundling.
Backend	Python / Flask	HTTP endpoints and business logic.
Database	MongoDB / PyMongo	Persistent application data.
Realtime	Flask-SocketIO / Socket.IO	Bidirectional event communication.
Authentication	JWT / PyJWT	Token-based protected access.
Password hashing	Flask-Bcrypt / bcrypt	Password hash support.
Production server	Gunicorn gthread	WSGI serving.
Hosting	Render	Separate Static Site and Web Service.
Crypto APIs	Web Crypto API	Browser cryptographic operations where implemented.

Logical architecture

Browser React client → HTTPS REST API → Flask application → MongoDB / GridFS. Browser client ↔ Socket.IO ↔ Flask-SocketIO. Browser crypto service performs supported encryption/decryption operations before and after message transport.

Deployment considerations

- Frontend and backend have independent build/deployment lifecycles.
- VITE_API_URL is embedded at frontend build time; rebuild after changing it.
- FRONTEND_URL controls backend CORS allowlisting.
- Socket.IO allowed origins must match the actual frontend origin.
- If backend state is in-memory, multi-instance scaling requires shared coordination/message queue; verify before scaling.

5. Frontend Design

File / module	Responsibility indicated by project inventory
App.jsx	Application root and top-level composition.
Landing.jsx / CSS	Public landing page.
Auth.jsx / CSS	Authentication interface.
Dashboard.jsx / CSS	Authenticated dashboard and navigation.
ChatWindow.jsx / CSS	Chat history, message send, sockets, crypto, image selection and chat controls.
PasswordRecovery.jsx	Password recovery flow.
RecoverySetup.jsx	Recovery setup flow.
IdentityRestore.jsx	Identity restoration workflow.
SignalTest.jsx / SocketTest.jsx	Test/diagnostic screens.
services/api.js	API base URL and shared HTTP request helper.
services/socket.js	Socket.IO connection using backend URL and auth token.
services/crypto.js	Key import/derivation and message encrypt/decrypt helpers.
services/signal/*	Signal client, relay, storage, device transfer and test modules.

API helper behavior

The shared API helper reads VITE_API_URL, attaches a bearer token when supplied, serializes JSON bodies, preserves FormData handling, parses JSON responses and raises errors for non-success responses.

UI quality requirements

- Responsive mobile, tablet and desktop layouts.
- Clear loading, empty, error and offline states.
- Accessible controls and keyboard-friendly selection.
- Do not expose tokens, plaintext or private keys in logs.
- Keep message history and bulk selection consistent across pagination.

6. Backend Design

Module	Responsibility indicated by module structure
app.py	Application initialization, configuration, CORS, MongoDB, GridFS, indexes, blueprints and Socket.IO.
extensions.py	Shared Flask-Bcrypt extension.
routes/auth.py	Authentication/account endpoints.
routes/users.py	User endpoints.
routes/connections.py	Connection/request endpoints.
routes/conversations.py	Conversation endpoints.
routes/messages.py	Message endpoints.
routes/signal_relay.py	Signal relay integration endpoints.
services/message_service.py	Message business logic.
socket_test.py	Socket test utility.

Startup/configuration

The backend expects SECRET_KEY and MONGODB_URI. It initializes MongoDB and performs a ping during startup. CORS is configured for the frontend origin; Socket.IO uses threading mode. Production serving is configured through Gunicorn.

Backend security responsibilities

- Validate JWTs on protected routes and socket connections.
- Authorize every conversation/message operation.
- Validate payloads, identifiers, attachment types and sizes.
- Enforce connection and block restrictions server-side.
- Avoid logging secrets, tokens, plaintext messages or private keys.
- Return safe errors without leaking stack traces or configuration.

MongoDB is the persistence layer. GridFS initialization appears in the project inventory. Exact collection names, fields, indexes, retention rules and access policies require confirmation from current backend source.

7. Data, API and Realtime Design

Logical entity	Information category
User	Account identifier, username/email, password hash, profile and account state.
Connection	Requester, recipient, status, timestamps and relationship state.
Conversation	Conversation ID, participant references and metadata.
Message	Conversation, sender, encrypted payload, timestamps and delivery/read metadata.
Identity/key record	Public identity key, device ID, fingerprint and key metadata.
Attachment	File metadata, storage reference, MIME type, size and authorization.

This is a conceptual entity catalogue, not a verified MongoDB schema.

API surface

Route modules indicate API groups for authentication, users, connections, conversations, messages and Signal relay. Confirmed endpoint examples include GET /api/health and the observed login request path /api/auth/login. Other exact paths and request/response schemas must be taken from route decorators and tests, not guessed.

Request lifecycle

- Client calls configured VITE_API_URL.
- API helper attaches JWT bearer token when required.
- Flask applies CORS and routes to the relevant blueprint.
- Backend validates identity, permission and payload.
- Backend reads/writes MongoDB/GridFS.
- Client updates state from response and authorized real-time events.

Realtime lifecycle

- Socket.IO connects to backend origin with auth token.
- Server validates connection and associates authenticated user.
- Authorized message/status events are emitted to participants.
- Clients reconcile socket events with REST history to avoid duplicates or stale state.

8. Security and Cryptographic Design

GRUS is positioned as a privacy-focused messaging application. Security claims must match implementation and verification evidence.

Controls represented in project context

- JWT-based API and socket authentication.
- Password hashing via bcrypt / Flask-Bcrypt.
- Client-side crypto service and Web Crypto API usage.
- Public-key import and shared conversation-key derivation functions.
- Message encryption before sending ciphertext in the described ChatWindow flow.
- Identity fingerprint display/verification and identity restore modules.
- CORS allowlisting between deployed frontend and backend.

Mechanism	Status in shared project information
AES-256-GCM	Listed as planned security design; confirm implementation and tests.
ECDH P-256	Listed as planned key agreement; verify key generation/import and agreement.
HKDF-SHA-256	Listed as planned key derivation; verify context binding and implementation.
Web Crypto API	Frontend crypto integration is present in project inventory.
Signal Protocol/relay	Signal client/relay/storage/device-transfer/test modules exist; production guarantees require verification.

Required security review

- Verify key generation, storage, backup, restore, rotation and revocation.
- Verify AES-GCM nonce uniqueness and authenticated decryption.
- Verify ECDH public-key validation and HKDF context separation.
- Verify identity-change warnings and fingerprint verification.
- Verify server does not receive plaintext/private keys if E2EE is claimed.
- Review metadata, logs, backups, attachments and notifications.
- Perform dependency scanning, static analysis, authorization testing and independent review.

Do not claim independently audited or fully end-to-end encrypted until the complete protocol and every relevant message/attachment path have been verified.

Secrets

- Keep SECRET_KEY, MONGODB_URI, mail credentials and provider keys in backend environment variables/secret manager.
- Never commit .env files or secrets.
- VITE_* values are visible in the built browser app; only public config belongs there.
- Rotate credentials exposed in chat, logs or source control.
- Use separate development and production secrets.

9. Deployment and Environment Configuration

Frontend Render Static Site

Setting	Value
Root Directory	frontend
Build Command	npm install && npm run build
Publish Directory	dist
VITE_API_URL	https://grus-messenger.onrender.com
VITE_OPEN_E2EE_RELAY_URL	Configured relay URL; public client config only.

Backend Render Web Service

Setting	Value
Root Directory	backend
Build Command	pip install -r requirements.txt
Start Command	gunicorn --worker-class gthread --threads 100 --bind 0.0.0.0:$PORT app:app
FRONTEND_URL	https://grus-429e.onrender.com (must match actual live origin)
SECRET_KEY	Set unique production secret in Render; never document its value.
MONGODB_URI	Set production MongoDB URI in Render; never document its value.
GRUS_ALLOW_DEV_PLAINTEXT	Set false only after confirming code behavior and encryption readiness.

Deployment sequence

- Commit and push reviewed source changes.
- Deploy backend and confirm service is Live.
- Confirm actual frontend URL and set same origin in backend FRONTEND_URL.
- Set frontend VITE_API_URL to backend URL and rebuild Static Site.
- Verify health endpoint, CORS preflight, login, socket and messaging.
- Check deployed commit hash in Render Events.

Frontend and backend are separate services. A local npm run build is a validation build; Render performs its own production build. Backend dependency installation is separate from the Vite build.

10. Installation and Developer Workflow

Prerequisites

- Python 3.10+ (use the version supported by project dependencies).
- Node.js and npm compatible with frontend dependencies.
- MongoDB instance and authorized connection string.
- Git and repository access.

Clone

```
git clone https://github.com/Aluval/grus-messenger.git
cd grus-messenger
```

Backend setup – Windows PowerShell

```
cd backend
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
# Configure backend environment variables
python app.py
```

Frontend setup

```
cd frontend
npm install
# Create .env with VITE_API_URL and public relay URL
npm run dev
```

Production build check

```
npm run build
```

A shared Render build log showed a successful Vite build with 2,470 modules transformed. It also reported four moderate npm audit findings and large-chunk warnings. These are not build failures; investigate safely and avoid forceful upgrades without compatibility review.

Git hygiene

- Exclude .env, virtual environments, node_modules, dist, logs and backups.
- Review git diff/status before committing.
- Never commit credentials, identity backups or private keys.
- Use focused commits and verify deployed commit hash.

11. Testing and Quality Assurance

The repository inventory includes crypto and Signal-related test files. This document does not claim that all tests currently pass.

Test level	Recommended checks
Unit	API helper serialization/errors; crypto round trip; key derivation; message merge and validation.
Backend integration	Registration/login; token validation; DB persistence; connection requests; authorization; message CRUD.
Frontend integration	Forms; dashboard; chat initialization; recovery and identity restore.
Realtime	Authenticated sockets; message delivery; read status; reconnect; duplicate handling; cross-user sync.
Security	Unauthorized access; IDOR; CORS; rate limiting; recovery abuse; upload validation; secret leakage.
Deployment	Health endpoint; static assets; API preflight; HTTPS; env values; restart/redeploy.
Compatibility	Responsive layouts and supported desktop/mobile browsers.

Critical end-to-end scenarios

- Register two test accounts and authenticate both.
- Create/accept a connection and start a one-to-one conversation.
- Send messages in both directions and verify real-time delivery.
- Reload and verify history retrieval and decryption.
- Test identity fingerprint and changed-key warning.
- Test block/unblock across REST and socket paths.
- Test delete-for-me, delete-for-everyone and clear conversation on both clients.
- Test attachment authorization and storage.
- Test invalid, expired and reused recovery tokens.

Release gate: authentication, authorization, messaging, deletion, CORS, socket behavior, persistence and advertised cryptographic properties must pass repeatable tests.

12. Monitoring, Operations and Troubleshooting

Monitor	URL	Purpose
GRUS Backend	https://grus-messenger.onrender.com/api/health	Checks API health response.
GRUS Frontend	https://grus-429e.onrender.com/	Checks static site availability.

A five-minute UptimeRobot interval was selected. Uptime checks only confirm endpoint availability; they do not prove login, database writes, socket messaging or encryption.

CORS troubleshooting

- FRONTEND_URL must exactly match browser origin including https and hostname.
- Avoid trailing slash in configured origin.
- Confirm CORS applies to /api/* and Socket.IO allowed origins.
- Confirm updated backend commit is deployed and Live.
- Inspect OPTIONS preflight response in browser Network tools.

Failed to fetch checklist

- VITE_API_URL points to backend, not frontend.
- Backend is live and /api/health responds.
- FRONTEND_URL matches actual frontend URL.
- Frontend rebuilt after VITE_* changes.
- Inspect Console/Network for CORS, DNS, TLS, timeout or server errors.

Operational controls

- Keep one backend instance unless shared state/socket scaling is configured.
- Monitor Render deployment events and logs.
- Back up MongoDB and test restore procedure.
- Review indexes, storage capacity and attachment retention.
- Rotate exposed credentials and review access permissions.

13. Risks and Open Verification Items

Priority	Item	Action
Critical	CORS origin drift	Confirm actual Render URL and synchronize FRONTEND_URL.
Critical	Secret hygiene	Rotate exposed credentials; audit Git history and Render environment.
Critical	Cryptographic assurance	Audit key lifecycle and all message/attachment paths.
High	Bulk deletion synchronization	Test pagination beyond latest 100 and cross-client consistency.
High	Authorization	Verify every REST and socket event checks participant/block state.
High	Database recovery	Document backup cadence, retention and restore testing.
Medium	Large frontend bundles	Apply code splitting/lazy loading and investigate large modules.
Medium	npm audit findings	Review four moderate findings and update safely.
Medium	Observability	Add structured logs and metrics without sensitive content.

Deployment observations

- Backend root URL returned a GRUS application status response.
- Health endpoint returned a healthy status response during checks.
- Frontend Vite production build completed successfully on Render.
- Browser previously reported a CORS preflight failure for POST /api/auth/login.
- Latest shared frontend URL is https://grus-429e.onrender.com; verify if Render changes it.

Update this list after retesting. It is not a declaration that all items remain unresolved.

14. Roadmap, Glossary and Ownership

Phase	Focus	Exit criteria
1 – Stabilize	CORS, env config, health monitoring, dependency review	Frontend/backend communication stable.
2 – Core messaging	Auth, connections, history, real-time delivery	Two-account E2E tests pass.
3 – Conversation controls	Selection, delete scopes, clear conversation, sync	Deletion semantics verified across clients/pagination.
4 – Security hardening	Crypto audit, identity verification, backup/restore	Review completed; claims match implementation.
5 – Production readiness	Backups, observability, performance, accessibility	Runbooks and release gate established.

Glossary

Term	Meaning
API	Application Programming Interface.
CORS	Cross-Origin Resource Sharing.
E2EE	End-to-end encryption.
JWT	JSON Web Token.
HKDF	HMAC-based Extract-and-Expand Key Derivation Function.
ECDH	Elliptic Curve Diffie-Hellman key agreement.
AES-GCM	Authenticated encryption mode.
Socket.IO	Event-based bidirectional communication library.
GridFS	MongoDB mechanism for storing larger files.

Copyright and licence

Copyright © 2026 Harsha Vardhan Goud. All Rights Reserved.

GRUS is intended to use a custom proprietary, all-rights-reserved notice. No permission is granted to copy, modify, reuse, redistribute, sublicense, sell, resell or commercially exploit the original project materials without prior written permission from the copyright owner. Third-party dependencies and assets remain subject to their own licences. Maintain a separate LICENSE file and seek legal review for commercial distribution.

Project links

GitHub: https://github.com/Aluval/grus-messenger
Frontend: https://grus-429e.onrender.com/
Backend: https://grus-messenger.onrender.com
Health: https://grus-messenger.onrender.com/api/health

GRUS — Private by Design. Connected in Real Time.